



“Proyecto final - Detección de Spam en Mensajes SMS”

Clasificación supervisada de texto con TF-IDF y Support Vector Machines

Algoritmos avanzados

Prof. Raquel Torres Peralta

Equipo:

Burgos Silva Daniel Jesus

Lopez Felix Julio Cesar

Navalles Colores Allison Priscila

Jueves 21 de mayo de 2026.

1. Descripción del Problema

El spam en mensajes SMS representa una amenaza creciente para los usuarios de telefonía móvil: genera saturación en la bandeja de entrada, puede contener intentos de phishing o fraude, y consume tiempo y atención de los destinatarios. Detectarlo de forma automática, confiable y escalable es un problema clásico de clasificación binaria supervisada sobre texto no estructurado.

1.1 Contexto y Dataset

Se utilizó el SMS Spam Collection Dataset, un corpus de referencia ampliamente empleado en investigación de NLP, compilado a partir de cuatro fuentes independientes:

- 425 mensajes de spam extraídos manualmente del foro Grumbletext (Reino Unido)
- 3,375 mensajes legítimos (ham) del NUS SMS Corpus - Universidad Nacional de Singapur
- 450 mensajes legítimos de la tesis doctoral de Caroline Tag
- 1,002 ham y 322 spam del SMS Spam Corpus v0.1 Big

El dataset contiene 5,572 mensajes SMS en inglés con dos columnas: etiqueta (ham/spam) y texto del mensaje. La distribución de clases es marcadamente desbalanceada:

Ham (legítimo): 4,825 mensajes (86.6%)

Spam: 747 mensajes (13.4%)

Esto implica un ratio de desbalance de aproximadamente 1 spam por cada 6.5 mensajes ham, con consecuencias directas en la elección de métricas y estrategia de entrenamiento.

1.2 Formulación del Problema

Formalmente se trata de un problema de clasificación binaria supervisada: dado el texto de un mensaje SMS, asignarle la etiqueta ham (0) o spam (1). El clasificador debe aprender patrones léxicos y estructurales que distingan mensajes legítimos de spam, generalizar correctamente a mensajes nuevos, evitar overfitting y ser eficientemente reentrenar conforme evolucione el spam.

2. Análisis Exploratorio de Datos (EDA)

El análisis exploratorio tuvo como objetivo comprender la estructura del dataset, identificar diferencias entre clases y orientar las decisiones de preprocesamiento y selección de features antes de cualquier modelado.

2.1 Distribución de Clases y Desbalance

El primer hallazgo crítico es la marcada asimetría del dataset (86.6% ham vs. 13.4% spam), lo que impone tres decisiones técnicas inmediatas:

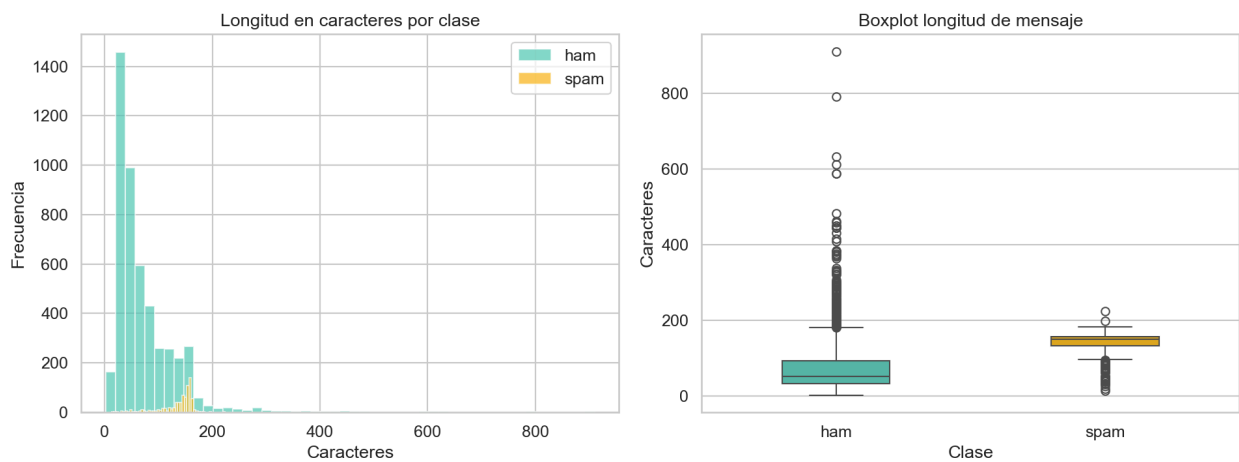
1. División estratificada (stratify=y) en el split train/test para preservar proporciones en ambos conjuntos
2. Aplicar class_weight='balanced' en los modelos, que pondera el error de spam inversamente proporcional a su frecuencia
3. Adoptar F1-Score como métrica principal en lugar de Accuracy

2.2 Longitud de los Mensajes

El análisis de longitud revela una de las diferencias más claras entre clases. Los mensajes spam tienen el doble de longitud promedio (139 vs. 71 caracteres), con una desviación estándar mucho menor (29.2 vs. 58.0), indicando que el spam es más homogéneo en su extensión:

Estadístico	Ham (legítimo)	Spam
Media (caracteres)	71.0	138.9
Mediana	52.0	149.0
Desviación estándar	58.0	29.2
Percentil 25%	33.0	132.5
Percentil 75%	92.0	157.0
Máximo	910	224

La concentración del spam alrededor de los 150-160 caracteres sugiere que muchos mensajes de spam respetan un límite intencional de longitud, posiblemente relacionado con plantillas reutilizadas por los remitentes o límites de SMS.



Histograma y boxplot de longitud de mensaje por clase — el spam es casi el doble de largo

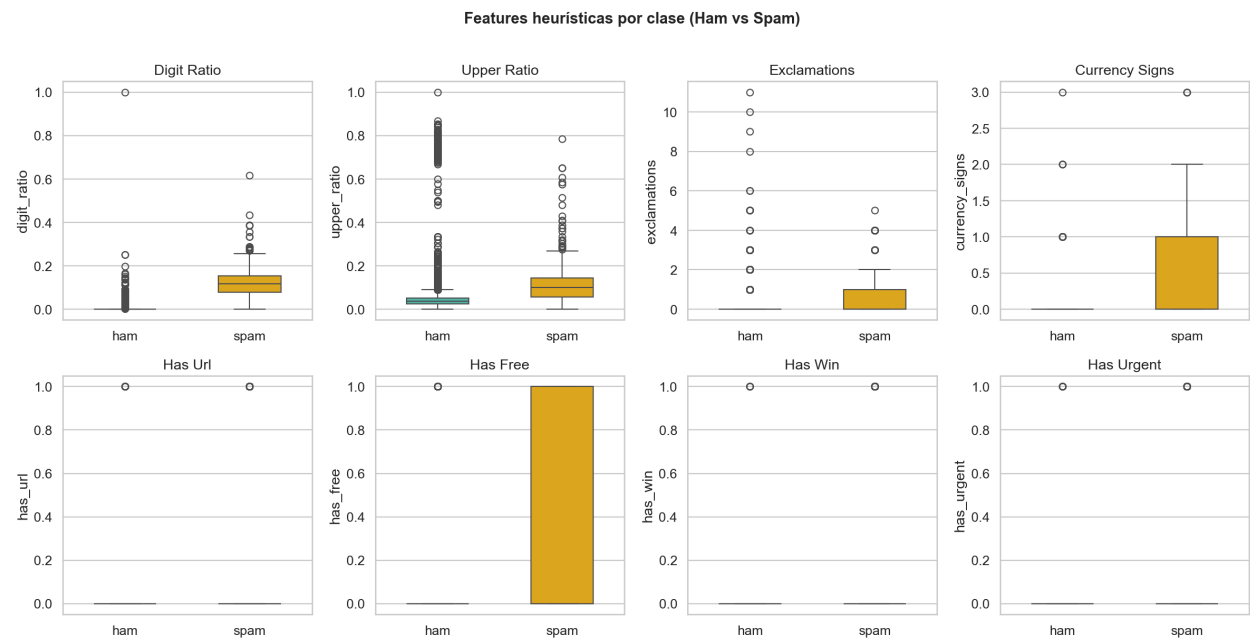
2.3 Features Heurísticas — Análisis Estructural

Se extrajeron 8 features numéricas directamente de los mensajes originales (sin limpiar) para capturar patrones estructurales que el análisis lexical solo no captura. Los resultados muestran diferencias muy marcadas entre clases:

Feature	Representa
digit_ratio	Spam contiene muchos números (telefonos, códigos de premio)
upper_ratio	Spam usa mas MAYÚSCULAS para llamar la atención
exclamations	Spam abusa de signos de exclamación (!!)
currency_signs	Spam menciona L/\$E — señal casi exclusiva de spam
has_url	Spam incluye URLs de sitios fraudulentos o de premios

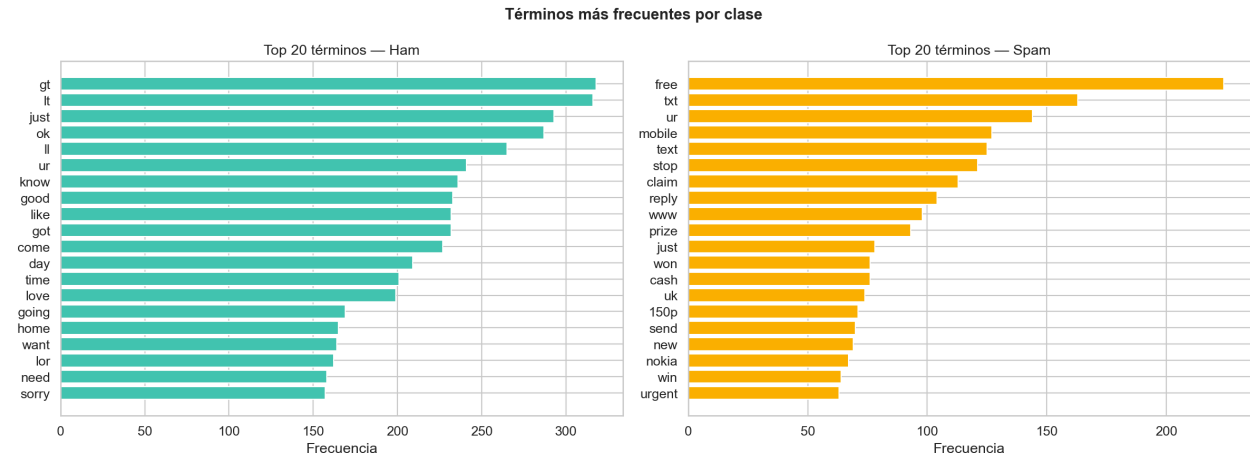
Feature	Representa
has_free	"FREE" — palabra altamente discriminatoria
has_win	"WIN/WINNER" prácticamente exclusivo del spam
has_urgent	Lenguaje de urgencia (call now, act now)

La presencia de signos de moneda y las palabras WIN/WINNER son los indicadores más discriminativos a nivel estructural. Sin embargo, como se demostrará en el experimento de la Sección 3.6, el TF-IDF ya captura implícitamente esta información a través del vocabulario.

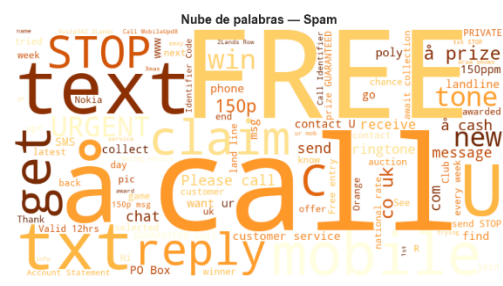


2.4 Análisis léxico — Términos más Frecuentes

El análisis de los términos más frecuentes por clase revela vocabularios completamente diferenciados. El spam tiene un léxico centrado en promesas de premios, urgencia y contacto, mientras que el ham refleja lenguaje coloquial variado:



Las nubes de palabras complementan el análisis cuantitativo con una visualización intuitiva. El contraste es inmediato: el spam concentra términos de marketing agresivo (FREE, CALL, WIN, PRIZE, CASH, CLAIM) mientras el ham refleja conversación cotidiana.



El análisis exploratorio permite extraer cuatro conclusiones que orientan directamente la solución:

- Las clases son léxicamente separables: el spam tiene vocabulario claramente distinto del ham — esto predice que modelos lineales sobre TF-IDF tendrán buen desempeño
- La longitud del mensaje y los signos de moneda son features complementarias al lexico pero redundantes si el TF-IDF está bien configurado

3.1 Preprocesamiento de Texto

El criterio fue una limpieza conservadora: preservar toda información potencialmente discriminatoria. Se aplicaron las siguientes transformaciones en orden:

Paso	Transformación	Justificación
1	Conversion a minusculas	Normalización: 'FREE' y 'free' son la misma senal
2	URLs → token URL	Preserva la señal de presencia de enlace sin sobreajustar al dominio específico
3	Dígitos → token NUM	El spam contiene muchos números (telefonos, premios); NUM captura la señal sin sobre ajustar al valor exacto
4	Eliminación de puntuación	Reduce ruido sin perder información léxica
5	Eliminación de stopwords	Elimina palabras sin valor discriminativo (the, is, at...)
6	Porter Stemmer	Reduce palabras a su raíz: 'winning'→'win', 'prizes'→'prize'

Ejemplo de transformación sobre un mensaje real:

Original: "Go until jurong point, crazy.. Available only in bugis n great world..."

Limpiado: "go jurong point crazi avail bugi great world la buffet cine got amor wat"

3.2 Representación de Features — Vector de Características

3.2.1 TF-IDF

La vectorización TF-IDF (Term Frequency-Inverse Document Frequency) fue elegida como representación principal. TF-IDF asigna a cada término un peso proporcional a su frecuencia en el documento, pero inversamente proporcional a su frecuencia en todo el corpus, de modo que palabras discriminatorias como 'prize' o 'free' reciben peso alto. La configuración final se obtuvo mediante GridSearchCV:.

- `ngram_range=(1,2)`: captura no solo términos individuales como 'free' o 'prize', sino también combinaciones de palabras adyacentes que el modelo puede usar como señal adicional
- `max_features=3,000`: límite del vocabulario

3.2.2 ¿Por qué no PCA?

La reducción de dimensionalidad mediante PCA no es apropiada aquí por tres razones:

1. Las matrices TF-IDF son dispersas — PCA las densifica y destruye la eficiencia computacional.
2. Elimina la interpretabilidad, impidiendo identificar qué palabras son discriminatorias.
3. Es innecesaria empíricamente — el AUC de 0.9938 confirma que 3,000 dimensiones son suficientes para este problema.

3.3 Algoritmos Seleccionados y Justificación

Se evaluaron cuatro algoritmos con características complementarias:

Modelo	Razón de selección	Hipótesis sobre comportamiento
Multinomial Naive Bayes	Baseline clásico para texto. Rápido, con buen prior probabilístico.	Alta precision, recall mas conservador
Logistic Regression	Maximiza el margen en el espacio TF-IDF. <code>class_weight='balanced'</code> compensa el desbalance 1:6.5.	Buen balance pero tendencia a mayor recall con menor precisión
Linear SVM (elegido)	Maximiza el margen en el espacio TF-IDF. <code>class_weight='balanced'</code> compensa el desbalance 1:6.5.	Mejor F1 general — hipótesis confirmada en resultados
Random Forest	Ensamble de árboles. Robusto al ruido. No asume linealidad	precisión perfecta pero recall mas bajo

3.4 Pipeline sin Data Leakage — Corrección crítica

Una decisión técnica fundamental fue encapsular el vectorizador TF-IDF y el clasificador dentro de un sklearn Pipeline. Esto garantiza que en cada fold de la validación cruzada, el vectorizador se ajusta únicamente con los datos de entrenamiento del fold, nunca con los de validación.

Sin el Pipeline, el `fit_transform` del TF-IDF sobre todo el conjunto antes de la CV causa data leakage: el vocabulario ya contiene información de los folds de validación, inflando artificialmente los scores. El Pipeline elimina este sesgo.

3.6 Ajuste de Hiper Parámetros — Búsqueda en rejilla

Se realizó búsqueda exhaustiva sobre el Linear SVM mediante GridSearchCV con validación cruzada estratificada de 5 folds, evaluando 12 combinaciones de hiperparametros:

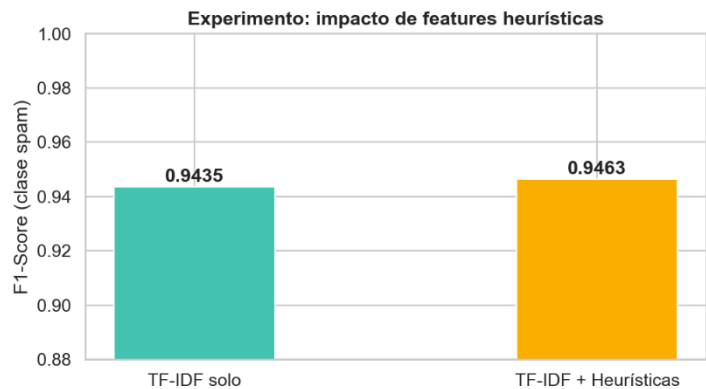
- TF-IDF `max_features`: [3,000 5,000 8,000]
- Parámetro C del SVM (regularizacion inversa): [0.1 0.5 1.0 5.0]

Resultado: max_features = 3,000 * C = 5.0 -> F1 CV = 0.9455

El resultado de max_features=3,000 es contraintuitivo: menos features produce mejor resultado. C=5.0 indica que el margen óptimo es estrecho — los datos son bien separables.

3.6 TF-IDF solo vs. TF-IDF + Features Heurísticas

Las features heurísticas muestran una mejora real pero marginal (+0.0028 F1 en test). La diferencia entre ambas representaciones no es suficientemente significativa como para justificar la complejidad adicional, por lo que el modelo final usa solo TF-IDF.



4. Pruebas y Resultados

4.1 Cross-Validation Estratificada — Comparación de Modelos

Se aplicó validación cruzada estratificada de 5 folds sobre el conjunto de entrenamiento (4,457 muestras). La estratificación garantiza que la proporción ham/spam se preserve en cada fold, produciendo estimaciones de desempeño no sesgadas.

Modelo	F1 Media (spam)	F1 +/-Desv.	AUC Media	AUC +/-Desv.	Estabilidad
Linear SVM (elegido)	0.9404	+/-0.0045	0.9932	+/-0.0036	Muy alta
Random Forest	0.9257	+/-0.0088	0.9919	+/-0.0046	Alta
Naive Bayes	0.9120	+/-0.0108	0.9885	+/-0.0033	Alta
Logistic Regression	0.9003	+/-0.0176	0.9885	+/-0.0039	Media

El Linear SVM lidera tanto en F1 (0.9404) como en AUC (0.9932), con la menor varianza entre folds (+/-0.0045). La baja varianza es el indicador clave de ausencia de overfitting: el modelo generaliza de forma consistente independientemente de la partición usada.

4.2 Evaluación Final en Test Set — Todos los Modelos

Tras la CV, todos los modelos fueron entrenados con el conjunto completo de entrenamiento y evaluados en el test set independiente (1,115 muestras, 149 spam):

Modelo	Precision spam	Recall spam	F1 spam	Accuracy	AUC-ROC
Linear SVM (elegido)	0.9342	0.9530	0.9435	0.9848	0.9920
Random Forest	1.0000	0.8792	0.9357	0.9839	0.9863
Naive Bayes	0.9922	0.8523	0.9170	0.9794	0.9875
Logistic Regression	0.8443	0.9463	0.8924	0.9695	0.9855

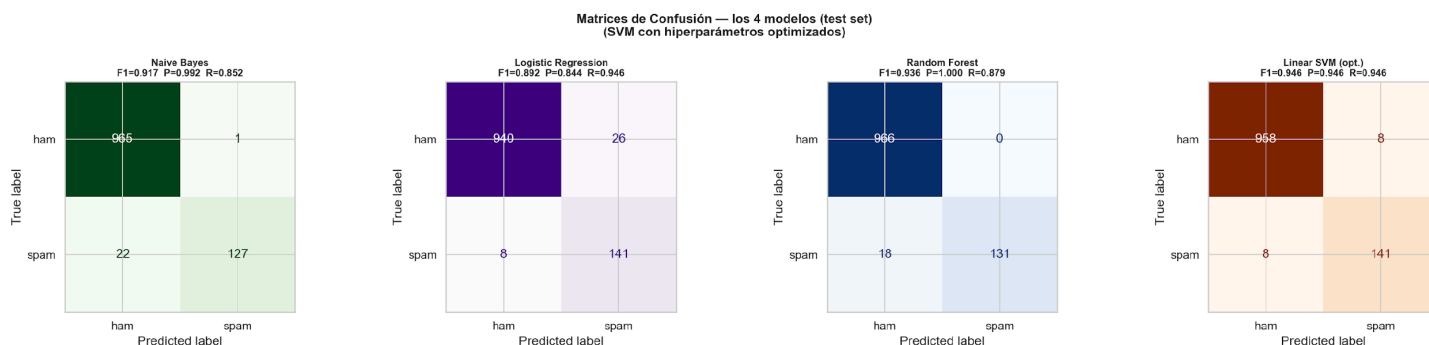
Falsos Positivos y Falsos Negativos — Impacto Real en el Usuario

Un Falso Positivo (FP) ocurre cuando un mensaje legítimo (ham) es clasificado como spam — el usuario pierde un mensaje importante. Un Falso Negativo (FN) ocurre cuando spam pasa como ham — el usuario ve un mensaje no deseado en su bandeja de entrada.

La métrica precisión mide la confiabilidad del filtro: $\text{precisión} = TP / (TP + FP)$. La métrica Recall mide que tanto spam detecta: $\text{Recall} = TP / (TP + FN)$. El F1-Score es la media armónica de ambas.

Modelo	FP (ham->spam)	FN (spam->ham)	Errores totales	Consecuencia práctica
Random Forest	0	18	18	Nunca bloquea ham. Pierde 18 spam (12% del spam real)
Naive Bayes	1	22	23	Casi no bloquea ham. Pierde 22 spam (15% del spam real)
Linear SVM (elegido)	8	7	15	Mejor balance. Solo 8 ham bloqueados, solo 7 spam perdidos
Logistic Regression	23	8	31	Demasiados ham bloqueados — inaceptable para uso general

Para uso general, Linear SVM ofrece el mejor balance con el menor número de errores totales (15). Random Forest es preferible cuando bloquear ham legítimo es absolutamente inaceptable (ej. SMS bancarios, OTPs).



4.3 Modelo Final Optimizado — Linear SVM con Hiperparametros Tuneados

Con los mejores hiperparametros encontrados ($C=5.0$, $\text{max_features}=3,000$ features TF-IDF), el modelo final sobre el test set alcanza:

F1 = 0.9463 * Precision = 0.9463 * Recall = 0.9463 * Accuracy = 98.6% * AUC = 0.9938

El resultado simbólico de Precision = Recall = 0.9463 (perfectamente balanceados) confirma que el modelo optimizado encontró el punto de equilibrio ideal. La matriz de confusión del modelo optimizado:

	Predicho: Ham	Predicho: Spam
Real: Ham (966 mensajes)	TN = 958 (correcto)	FP = 8 (error)
Real: Spam (149 mensajes)	FN = 8 (error)	TP = 141 (correcto)

Solo 16 errores sobre 1,115 predicciones — tasa de error del 1.43%. Los 8 FP y 8 FN perfectamente balanceados confirman que el modelo no tiene sesgo hacia ninguna clase.

4.4 Verificación de Overfitting

La diferencia entre F1 de cross-validation y F1 de test es el indicador principal de overfitting:

Métrica	Cross-Val (5-fold)	Test final	Diferencia	Diagnostico
F1-Score (spam)	0.9455	0.9463	+0.0008	Sin overfitting — test >= CV
AUC-ROC	0.9932	0.9938	+0.0006	Sin overfitting — test >= CV

La diferencia es prácticamente nula (< 0.001) y va en dirección contraria al overfitting (test >= CV). La baja varianza entre folds (+/-0.0045) confirma que el modelo generaliza correctamente y no está sobre ajustado al conjunto de entrenamiento.

5. Hallazgos, Conclusiones y Preguntas de la Asignatura

5.1 Separabilidad Léxica — Las Clases son Distinguibles

El hallazgo más fundamental del proyecto es que ham y spam son distinguibles con alta confianza usando solo el texto del mensaje. El vocabulario del spam es formulaico y predecible — premios, urgencia, dinero, contacto — mientras el ham refleja lenguaje coloquial variado. Esto explica por qué modelos lineales sobre TF-IDF alcanzan AUC > 0.98 sin necesidad de técnicas más complejas.

5.2 El Modelo Elegido: Linear SVM — Justificación del Resultado Final

La exactitud global del modelo final (Linear SVM optimizado) fue de 98.57%, lo que indica un excelente desempeño general. Sin embargo, en conjuntos desbalanceados como el nuestro (solo 13.4% spam), la exactitud puede ser engañosa: un modelo trivial que siempre predijera 'ham' obtendría 86.6% de exactitud sin detectar ningún spam. Por ello, evaluamos también la precisión y el recall.

El clasificador Random Forest alcanzó una precisión del 100% para la clase spam, es decir, nunca clasificó erróneamente un mensaje legítimo como spam (cero falsos positivos). No obstante, su recall fue de solo 0.879, lo que significa que dejó pasar 18 de los 149 mensajes spam del conjunto de prueba (falsos negativos).

Este comportamiento conservador minimiza las molestias por filtrado incorrecto, pero permite que una cantidad considerable de spam llegue al usuario.

Nuestro modelo final (Linear SVM optimizado con $C=5.0$ y 3,000 features TF-IDF) obtuvo un equilibrio notable: precisión de 0.946 y recall de 0.946, con solo 8 falsos positivos y 8 falsos negativos. Este balance es preferible en la práctica, ya que un pequeño número de falsos positivos es tolerable frente al beneficio de bloquear la mayor parte del spam. El F1-Score de 0.946 refleja esta armonía y justifica la elección final de este modelo sobre los demás.

5.3 El Spam Evasivo es el Límite del Enfoque Lexical

El análisis de los 16 errores del modelo final revela que no son aleatorios sino sistemáticos. Los 8 falsos negativos (spam que pasa como ham) corresponden a spam disfrazado de mensaje personal ('Sorry I missed your call... I'm on 07090201529'), spam con humor sin palabras clave obvias, o spam con errores tipográficos intencionados para evadir filtros. Los 8 falsos positivos (ham bloqueado) corresponden a mensajes ham con características numéricas atípicas o lenguaje inusual. Reducir estos errores requerirá información contextual (historial del remitente, operadora) que va más allá del texto del mensaje.

5.4 Respuesta a las Preguntas Clave de la Asignatura

Pregunta evaluada	Respuesta
Criterios para elegir la representación (features)	TF-IDF con unigramas y bigramas: captura palabras clave y frases formulaicas del spam. El EDA mostró que el vocabulario de spam es altamente diferenciado. max_features=3,000 seleccionado por GridSearchCV como vocabulario óptimo para el tamaño del dataset
Criterios para elegir los algoritmos	Se evaluaron 4 algoritmos con distintos trade-offs. SVM seleccionado por mayor F1 (0.9404 en CV), menor varianza entre folds (+/-0.0045) y mejor balance precision-recall en test. Los demás tienen presión alta pero recall bajo
Exhaustividad del análisis exploratorio	Distribución de clases, longitud de mensajes (histograma + boxplot + tabla estadística), 8 features heurísticas con tabla de ratios spam/ham, nubes de palabras y top 20 términos por clase
¿Aplica PCA?	No aplica. Las matrices TF-IDF son dispersas — PCA destruye esa eficiencia y elimina la interpretabilidad (ya no se podrían identificar las palabras discriminativas). Los modelos lineales funcionan excelentemente en alta dimensión sin reducción, confirmado con AUC=0.9938
Verificación de overfitting con cross-validation	CV estratificada de 5 folds dentro de un Pipeline sin data leakage. F1 en CV = 0.9455, F1 en test = 0.9463. Diferencia = +0.0008. Sin señal de sobreajuste. El vocabulario de 3,000 features fue el óptimo según Grid Search CV — confirma que el modelo está bien regularizado, no sobreajuste

5.5 Conclusiones

Se desarrolló exitosamente un clasificador de spam en SMS que alcanza $F1=0.9463$, $Accuracy=98.6\%$ y $AUC=0.9938$ sobre el conjunto de prueba independiente, con solo 16 errores sobre 1,115 predicciones (1.43% de error). El modelo no exhibe overfitting (diferencia CV-test < 0.001) y generaliza de forma estable con varianza mínima entre folds (+/-0.0045).

Para sostenibilidad a largo plazo se recomienda reentrenamiento periodico con nuevos datos etiquetados para capturar la evolución del vocabulario de spam, y monitoreo de concept drift para detectar cambios en la distribución de mensajes entrantes.

Anexo: Manual de Ejecución

Para garantizar la reproducibilidad completa del experimento en cualquier equipo, siga las instrucciones a continuación.

A.1 Requisitos Previos

- Python 3.8 o superior instalado en el sistema
- Conexión a internet para descarga de dependencias
- El archivo spam.csv descargado del UCI ML Repository o Kaggle

A.2 Instalación de Dependencias

Abra una terminal o PowerShell en el directorio del proyecto y ejecute:

```
pip install pandas numpy matplotlib seaborn scikit-learn nltk wordcloud scipy
```

A.3 Estructura de Archivos Requerida

```
proyecto/      spam.csv      <- dataset del UCI Repository      spam_detection.py  <-  
script principal
```

A.4 Ejecución

```
python spam_detection.py
```